

Solveur Easy as ABC

Projet LOMA

Arthur & Maxence - ENSIE FISA 1A

13 mai 2026

1 Introduction

Ce document présente les choix d'architecture et a pour objectif d'aider à comprendre la démarche mise en place dans le projet ainsi que des exemples d'utilisation. Pour des exemples d'implémentation, veuillez vous référer au fichier README, qui contient des illustrations plus précises et directement exploitables.

1.1 Présentation du Projet

Ce projet consiste en la réalisation d'un solveur pour le puzzle **Easy as ABC**. L'objectif du jeu est de remplir une grille avec des lettres de sorte que chaque lettre apparaisse une seule fois par ligne et par colonne, tout en respectant les indices situés sur les bords ou dans la grille.

1.2 Objectif

Notre objectif est de coder et d'implémenter un programme capable de résoudre un puzzle à partir d'un input. En utilisant un solveur SAT (Glucose), le programme transforme le problème en un format adapté au solveur, puis récupère et interprète la solution afin de retourner le puzzle résolu.

2 Choix Technologiques

Nous avons choisi le langage C, car c'est un langage que nous avons beaucoup utilisé ces derniers temps et que nous maîtrisons bien, ce qui nous permet de générer un exécutable à partir du code. Pour les tests et la validation, nous avons utilisé Python afin d'accélérer le développement grâce à sa simplicité pour exécuter et tester l'exécutable fait en C.

3 Analyse et Conception

Cette section détaille la démarche intellectuelle suivie pour transformer les règles du puzzle "Easy as ABC" en un problème solvable par un outil SAT. Notre conception s'articule autour de trois axes principaux : l'analyse des contraintes du jeu, la modélisation de ces contraintes sous forme de clauses logiques, et l'architecture globale du programme qui orchestre le processus.

3.1 Analyse des Contraintes du Puzzle

Avant toute implémentation, nous avons décomposé le puzzle en un ensemble de règles formelles. Pour une grille de taille $N \times N$ avec M lettres distinctes :

Contraintes :

- Chaque case de la grille doit contenir soit une lettre parmi les M possibles, soit rester vide.
- Chaque lettre ne doit apparaître qu'une seule fois par ligne.
- Chaque lettre ne doit apparaître qu'une seule fois par colonne.
- Les lettres initialement présentes dans la grille d'entrée doivent être conservées dans la solution.
- Pour chaque indication placée sur le bord de la grille, la lettre correspondante doit être la première visible depuis ce bord dans la solution.

3.2 Modélisation du Problème pour un Solveur SAT

La phase cruciale de notre projet a été de traduire ces règles en une formule de logique propositionnelle en Forme Normale Conjonctive (FNC), le format attendu par les solveurs SAT comme Glucose.

3.2.1 Définition des Variables Booléennes

Le cœur de la modélisation repose sur le choix imposé de variables : Soit la variable $x_{i,j,k}$ qui est **vraie** si et seulement si la case à la ligne i et la colonne j contient la lettre k .

- $i \in \{1, \dots, N\}$ (ligne)
- $j \in \{1, \dots, N\}$ (colonne)
- $k \in \{A, \dots, M\}$

Pour une grille de taille 5×5 avec 4 lettres, nous avons donc $5 \times 5 \times 4 = 100$ variables booléennes.

3.2.2 Traduction des Règles en Clauses FNC

1. **Au moins une lettre par case :**

$$(x_{i,j,1} \vee x_{i,j,2} \vee \dots \vee x_{i,j,M})$$

2. **Au plus une lettre par case :**

$$(\neg x_{i,j,k_1} \vee \neg x_{i,j,k_2}) \quad \forall k_1 \neq k_2$$

3. **Pour chaque ligne i et chaque lettre k , cette lettre doit apparaître au plus une fois :**

$$(x_{i,1,k} \vee x_{i,2,k} \vee \dots \vee x_{i,N,k})$$

Même chose pour les colonnes.

4. **Cases pré-remplies :** Si la case (i, j) contient initialement la lettre k :

$$(x_{i,j,k})$$

5. **Indices de visibilité :** Un indice C à gauche de la ligne i . Cela signifie que la première lettre rencontrée est C :

- La case $(i, 1)$ contient C . OU
- La case $(i, 1)$ ne contient pas de lettre ET la case $(i, 2)$ contient C . OU
- etc.

3.3 Architecture et Flux d'Exécution du Programme

Le programme en C est conçu pour pouvoir trouver au maximum deux solutions si elles existent.

1. Étape 1 : Parsing de l'Entrée

- Le programme lit le fichier texte du puzzle fourni en argument.
- Il extrait la taille de la grille N , le nombre de lettres M , les indices sur les bords et les cases pré-remplies.
- Ces informations sont stockées dans des structures de données internes telles que les struct `sat` et `puzzle` voir `sat.c` et `puzzle.c`.

2. Étape 2 : Génération du Fichier DIMACS

- À partir des données parsées, le programme construit une représentation de l'ensemble des contraintes du puzzle et l'écrit dans un fichier temporaire (`problem.cnf`) en respectant le format standard DIMACS, le format d'entrée de Glucose.

3. Étape 3 : Appel du Solveur SAT

- Le programme C invoque le solveur Glucose via un appel système (`system()`).
- L'appel système est structuré pour séparer le statut du résultat de la solution, ce qui facilite le parsing ultérieur. La commande est :

```
chemin/vers/glucose problem.cnf > statut.txt solution.txt
```

- `> statut.txt` capture les statistiques d'exécution du solveur.
- `solution.txt` est interprété par Glucose comme le fichier de destination où il doit écrire la liste des règles qui rendent la formule vraie. Cette séparation des sorties nous a permis de gérer de manière robuste la lecture du résultat.

4. Étape 4 : Interprétation de la Solution

- Le programme attend la fin de l'exécution de Glucose, puis lit le fichier de résultat.
- le programme lit la liste des littéraux vrais qui constituent la solution. Chaque littéral positif est reconverti dans notre système de coordonnées (i, j, k) pour déterminer quelle lettre va dans quelle case.

5. Étape 5 : Affichage et Recherche de Solutions Multiples

- La grille solution est construite et affichée sur la sortie standard et enregistrée dans les structures de données.
- Pour vérifier l'unicité de la solution, une étape supplémentaire peut être enclenchée :
 - (a) **Négation de la solution** : Une nouvelle clause est générée. Cette clause est la négation de la solution qui vient d'être trouvée.
 - (b) **Nouvel appel** : Cette clause de négation est ajoutée au fichier DIMACS original, et le solveur est rappelé (retour à l'étape 3).
 - (c) **Conclusion** : Si ce nouvel appel retourne `UNSATISFIABLE`, cela prouve que la première solution trouvée était unique. S'il retourne `SATISFIABLE`, cela signifie qu'il existe au moins une autre solution afficher aussi.

4 Présentation du Format choisi

Format de Fichier

Les instances du puzzle sont définies dans un fichier texte respectant le format suivant :

```
# Puzzle
5 4
  A C      D
. . . . .D
B. . . . .
. . . . .B
. . . . .C
. . . . .
  D C
```

Dans l'exemple :

- 5 correspond à une grille de taille 5×5 ,
- 4 indique qu'il y a 4 lettres par ligne et par colonne.

Description du format

- La première ligne contient un caractère # suivi du nom du puzzle.
- La deuxième ligne contient :
 - la taille de la grille,
 - puis le nombre de lettres du puzzle.
- Le contenu de la grille est représenté par des caractères :
 - . représente une case vide,
 - une lettre représente une case déjà remplie.
- Les caractères à l'intérieur de la grille sont séparés par un espace uniquement horizontalement.
- La grille doit être décalée d'un espace par rapport au bord gauche.
- Des lettres peuvent être placées autour de la grille pour représenter les indices de visibilité. Un espace indique l'absence d'indice.
- Toutes les lettres doivent être écrites en majuscules.

5 Tests en Python

Pour garantir la robustesse et la correction de notre solveur en C, nous avons développé une suite de tests en Python. Cette approche nous a permis d'automatiser la validation de notre programme en tirant parti de la simplicité de Python pour la gestion des fichiers et de son exécution.

Le principe est simple : un script Python exécute notre programme C compilé avec un fichier de puzzle en entrée, capture sa sortie, et la compare à une solution attendue.

Cette méthode nous a permis de constituer rapidement une batterie de tests couvrant de multiples cas :

- **Tests de non-régression** : Après chaque modification du code C, nous pouvions relancer tous les tests pour nous assurer que rien n'avait été cassé.
- **Validation de cas limites** : Il était facile de créer des fichiers de puzzles pour des cas spécifiques (Easy 1, Easy 2, etc.).
- **Logique correcte** : Il nous a aussi permis de vérifier que la logique implémentée pour le solveur était bien la bonne (ce qui n'était pas le cas du premier coup).

6 Résultats

Pour démontrer l'efficacité et la correction de notre solveur, nous présentons ici les résultats obtenus sur plusieurs instances de puzzles. Les exemples illustrent différents scénarios : un cas de résolution standard, un puzzle sans solution, et un cas avec plusieurs solutions possibles.

6.1 Exemple 1 : Résolution d'un puzzle standard

Cet exemple montre la capacité du solveur à résoudre un puzzle ayant une solution unique.

Puzzle en entrée (puzzle5.data)

Sortie du programme

```
PS D:\Project Informatique\miniprojet-LOMA> .\bin\puzzle_solver.exe -pi .\input\puzzle2.data

----- Input -----

Puzzle size = 5
Puzzle nb letters = 5
  A C      D
  - - - - -
B | . . . . | D
  | . . . . | E
  | . . . . | B
  | . . . . | C
E | . . . . |
  - - - - -
    D C

-----

>>> Finding puzzle solution ...

SAT !
Solution:
A C B E D
B A D C E
C E A D B
D B E A C
E D C B A

>>> Trying to find second solution ...

UNSATISFIABLE

Results: 1 solution found !
```

FIGURE 1 – Résolution du puzzle standard

6.2 Exemple 2 : Puzzle avec plusieurs solutions

Ce cas illustre la capacité de notre programme à gérer les puzzles non uniques. Après avoir trouvé une première solution, il la nie et relance le solveur afin d'en trouver une seconde.

Puzzle en entrée (puzzle6.data)

Sortie du programme

```
PS D:\Project Informatique\miniprojet-LOMA> .\bin\puzzle_solver.exe -pi .\input\puzzle6.data
----- Input -----
Puzzle size = 8
Puzzle nb letters = 7

  - - - - -
  | . . . . . | A
  | . . . . . |
  | . . . . . |
  | . . . . . |
  | . . . . . | B
  | . . . . . |
  | . . . . . |
  | . . . . . |
  - - - - -
C |
  |
  |
  |
  |
  |
  |
  |
  - - - - -

>>> Finding puzzle solution ...

SAT !
Solution:
_ C D B E F G A
A D B _ C E F G
D B C A _ G E F
B _ A F G C D E
E A _ G F B C D
F E G C A D _ B
C G F E D A B _
G F E D B _ A C

>>> Trying to find second solution ...

SAT !
Solution:
B C D _ E F G A
_ D B A C E F G
D A C B _ G E F
A B _ F G C D E
E _ A G F B C D
F E G C A D _ B
C G F E D A B _
G F E D B _ A C

Results: 2 solutions found :D
```

FIGURE 2 – Exemple d'un puzzle possédant plusieurs solutions

6.3 Exemple 3 : Puzzle sans solution (UNSAT)

Puzzle en entrée (puzzle1.data)

Sortie du programme

```
PS D:\Project Informatique\miniprojet-LOMA> .\bin\puzzle_solver.exe -pi .\input\puzzle1.data

----- Input -----

Puzzle size = 4
Puzzle nb letters = 4
      B
  - - - -
  | . . . . |
  | . . . . | C
C | . . A . |
A | . . . . |
  - - - -
      A

-----

>>> Finding puzzle solution ...

UNSATISFIABLE

Results: No solution found :(
```

FIGURE 3 – Détection d'un puzzle sans solution

Ces résultats valident notre approche : le programme est capable de résoudre des puzzles, de détecter les cas d'impossibilité et de gérer les solutions multiples, conformément aux objectifs fixés.